Algorithms

Topic	Algorithms to Master
Arrays	Two Pointers, Sliding Window, Kadane's Algorithm
Sorting	Quick Sort, Merge Sort (Understand their time/space complexity)
Searching	Binary Search (The most important searching algorithm)
Trees	DFS (Pre/In/Post-order) and BFS (Level Order)
Graphs	BFS, DFS, Topological Sort (Kahn's Algorithm)
Linked Lists	Floyd's Cycle-Finding (Tortoise and Hare)
DP	0/1 Knapsack, Longest Common Subsequence (LCS)
Bitwise	XOR operations (for "Single Number" problems)

2. Second Priority

- **Dijkstra's Algorithm:** Very common for graph-heavy.
- **Heap Sort:** Usually, you just need to know how to use a Priority Queue (Min-Heap/Max-Heap) rather than implementing the sort itself.
- **Union-Find (DSU):** Crucial for "Number of Islands" variations and Kruskal's.
- **Trie (Prefix Tree):** Not in your list, but highly recommended for string search problems.
- **Backtracking:** Not an algorithm per se, but the "Template" for N-Queens and Sudoku is vital.

3. Non-Important / Low-Priority

- **Manacher's Algorithm:** Extremely rare. If a problem requires this, there is almost always a DP or Sliding Window approach that is "good enough" for an interview.
- **KMP / Rabin-Karp:** Most interviewers are satisfied if you explain the *concept* of rolling hashes or preprocessing. Unless you are interviewing for a role in compilers or specialized text processing, memorizing the KMP failure function is overkill.
- **Ternary Search:** Rarely comes up compared to Binary Search.
- **Bellman-Ford:** Only necessary if the graph has negative weights. Dijkstra is 10x more likely.
- **Huffman Coding:** More of a "System Design" or "Compression" concept; rarely asked as a coding challenge.
- **Counting Sort:** Only useful in very specific constraints; usually, a hash map (Frequency Map) covers this logic more flexibly.
- **Hash Map / Frequency Map** (The "cheat code" for most problems).

Core Complexity Cheat Sheet

Algorithm / Data Structure	Best Time	Average Time	Worst Time	Space Complexity
Binary Search	O(1)	O(log n)	O(log n)	O(1)
Quick Sort	O(n \log n)	O(n \log n)	O(n^2)	O(log n)
Merge Sort	O(n \log n)	O(n \log n)	O(n \log n)	O(n)
BFS / DFS (Graph)	—	O(V + E)	O(V + E)	O(V)
Dijkstra's	—	O(E \log V)	O(E \log V)	O(V)
Hash Map Lookup	O(1)	O(1)	O(n)	O(n)

1. Searching & Sorting (The Foundation)

Binary Search is the most frequently tested algorithm. Interviewers love it because it tests your ability to handle "off-by-one" errors.

- **Tip:** Always check your `left <= right` condition and how you update `mid`.
- **Merge Sort vs. Quick Sort:** Know that Merge Sort is **stable** (preserves order of equals) but uses more memory (O(n)), while Quick Sort is often faster in practice but has a O(n^2) worst-case.

2. Graph Traversals (The "Swiss Army Knife")

Most "Grid" problems (like finding an island or a path) are just Graph problems in disguise.

- **BFS (Breadth-First Search):** Use this when asked for the **shortest path** in an unweighted graph. It uses a **Queue**.
- **DFS (Depth-First Search):** Use this for exhaustive searches or exploring all possibilities (Backtracking). It uses a **Stack** (or Recursion).

3. The Sliding Window (The Optimization King)

Whenever you see a problem asking for a "sub-array" or "sub-string" with a specific property (e.g., "longest sub-string with 3 unique characters"), think **Sliding Window**.

- **Fixed Window:** Use a simple loop.
- **Dynamic Window:** Use two pointers (`left` and `right`) and a Hash Map to track frequencies. This turns a O(n^2) problem into a O(n) problem.

4. Dynamic Programming (The Pattern Matcher)

Don't try to memorize every DP problem. Instead, learn these two patterns:

1. **0/1 Knapsack:** "Can I reach this sum using these elements?"
2. **Longest Common Subsequence:** Comparing two strings or sequences.

Data Structures

1. Arrays & Strings

Focus: Two pointers, sliding window, and frequency maps.

Topic	Classic Interview Problems
Arrays	Two Sum, Best Time to Buy/Sell Stock, Product of Array Except Self, Merge Intervals
Strings	Valid Anagram, Longest Substring Without Repeating Characters, Palindrome Partitioning, String to Integer (atoi)

2. Linked Lists

Focus: Pointer manipulation and cycle detection.

- **Key Techniques:** Fast and Slow Pointers (Tortoise and Hare), Dummy Nodes, Reversing in-place.
- **Essential Problems:**
  - Reverse a Linked List (Iterative & Recursive)
  - Detect Cycle in a Linked List (Floyd's Cycle-Finding)
  - Merge Two Sorted Lists
  - Remove N-th Node From End of List
  - Copy List with Random Pointer

3. Searching & Sorting

Type	Must-Solve Problems
Searching	Search in Rotated Sorted Array, Find Minimum in Rotated Sorted Array, Median of Two Sorted Arrays
Sorting	Sort Colors (Dutch National Flag), K-th Largest Element in an Array, Merge Sorted Arrays

4. Stacks & Queues

Focus: LIFO/FIFO logic and Monotonic sequences.

- **Key Concepts:** Monotonic Stack, Circular Queue, Priority Queue (Heaps).
- **Essential Problems:**
  - Valid Parentheses (Stack)
  - Daily Temperatures (Monotonic Stack)
  - Implement Queue using Stacks
  - Sliding Window Maximum (Deque)
  - Min Stack

5. Trees & Graphs

Focus: Traversal (BFS/DFS) and pathfinding.

Topic	Essential Problems
Trees	Maximum Depth of Binary Tree, Validate BST, Lowest Common Ancestor (LCA), Binary Tree Level Order Traversal
Graphs	Number of Islands, Course Schedule (Topological Sort), Word Ladder, Network Delay Time (Dijkstra)

6. Recursion & Backtracking

Focus: Exploring state spaces and decision trees.

- **Key Concepts:** Base cases, pruning, state recovery.
- **Essential Problems:**
  - Generate Parentheses
  - Permutations / Combinations
  - N-Queens Problem
  - Sudoku Solver
  - Word Search

7. Dynamic Programming (DP)

Focus: Overlapping subproblems and optimal substructure.

- **Key Patterns:** 0/1 Knapsack, Longest Common Subsequence (LCS), Longest Increasing Subsequence (LIS), Fibonacci-style.
- **Essential Problems:**
  - Climbing Stairs
  - Coin Change
  - Longest Palindromic Substring
  - House Robber

    - Edit Distance

8. Greedy Algorithms

Focus: Making the locally optimal choice at each step.

- **Key Algorithms:** Huffman Coding, Activity Selection, Fractional Knapsack.
- **Essential Problems:**
    - Jump Game
    - Gas Station
    - Non-overlapping Intervals
    - Task Scheduler

9. Bit Manipulation

Focus: Understanding binary representations and bitwise operators (`&`, `|`, `^`, `~`, `<<`, `>>`).

- **Key Techniques:** Checking if a bit is set, toggling bits, clear rightmost set bit (`n & (n-1)`).
- **Essential Problems:**
    - Number of 1 Bits (Hamming Weight)
    - Single Number (using XOR)
    - Counting Bits
    - Reverse Bits